Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3
По дисциплине: «Современные платформы программирования»

Выполнил:
Студент 3 курса
Группы ПО-12
Ишимов А.В.
Проверила:
Кулик А.Д.

Брест 2026

Вариант 7

Цель работы: приобрести навыки применения паттернов проектирования при решении практических задач с использованием языка Python.

Задание 1. Прочитать задания, взятые из каждой группы, соответствующей одному из трех основных типов паттернов; Определить паттерн проектирования, который может использоваться при реализации задания. Пояснить свой выбор; Реализовать фрагмент программной системы, используя выбранный паттерн. Реализовать все необходимые дополнительные классы.

Преподаватель. Класс должен обеспечивать одновременное взаимодействие с несколькими объектами класса Студент. Основные функции преподавателя: Проверить Лабораторную Работу, Провести Консультацию, Принять Экзамен, Выставить Отметку, Провести Лекцию.

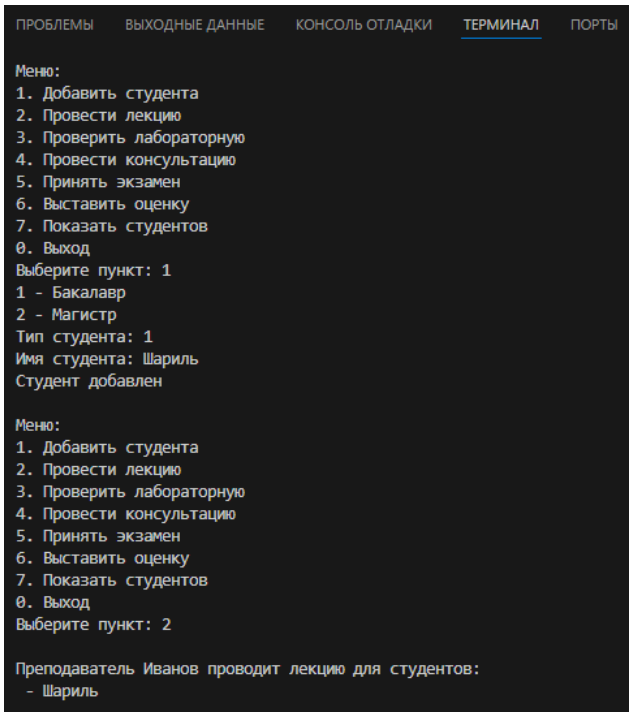
Код программы

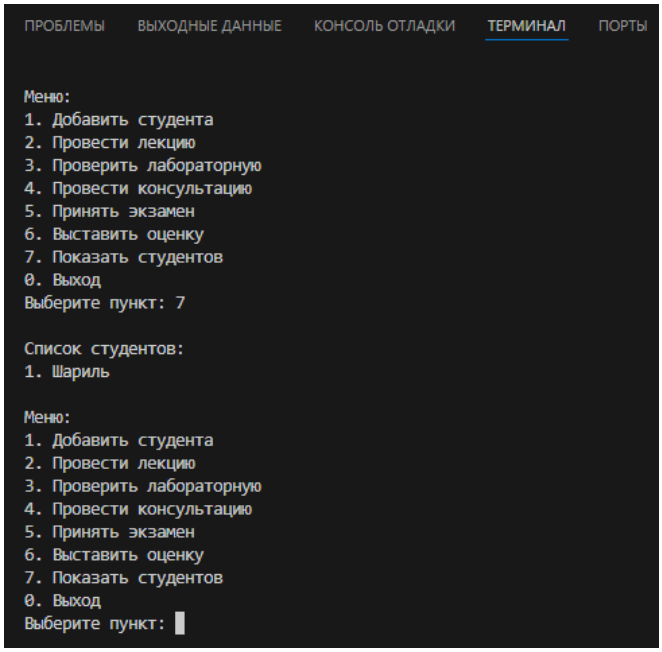
```
from abc import ABC, abstractmethod


# Абстрактный класс Студент
class Student(ABC):

    def __init__(self, name):
        self.name = name
        self.grade = None

    @abstractmethod
    def study(self):
        pass


# Конкретные классы студентов
class BachelorStudent(Student):

    def study(self):
        print(f"Бакалавр {self.name} изучает базовый курс.")


class MasterStudent(Student):

    def study(self):
        print(f"Магистр {self.name} изучает углубленный курс.")


# Фабрика студентов (Factory Method)
class StudentFactory:
```

```
    @staticmethod
    def create_student(student_type, name):
        if student_type == "1":
            return BachelorStudent(name)
        elif student_type == "2":
            return MasterStudent(name)
        else:
            raise ValueError("Неизвестный тип студента")


# Класс Преподаватель
class Teacher:

    def __init__(self, name):
        self.name = name
        self.students = []

    def add_student(self, student):
        self.students.append(student)
        print("Студент добавлен")

    def check_lab(self, student):
        print(f"Преподаватель {self.name} проверяет лабораторную работу студента {student.name}")

    def consult(self, student):
        print(f"Преподаватель {self.name} проводит консультацию со студентом {student.name}")

    def exam(self, student):
        print(f"Преподаватель {self.name} принимает экзамен у студента {student.name}")

    def grade(self, student, grade):
        student.grade = grade
        print(f"Преподаватель {self.name} выставил оценку {grade} студенту {student.name}")

    def lecture(self):
        print(f"\nПреподаватель {self.name} проводит лекцию для студентов:")
        for student in self.students:
            print(f" - {student.name}")

    def show_students(self):
        if not self.students:
            print("Нет студентов")
            return

        print("\nСписок студентов:")
        for i, student in enumerate(self.students):
            print(f"{i + 1}. {student.name}")


# Главное меню
def main():

    teacher = Teacher("Иванов")

    while True:
        print("\nМеню:")
        print("1. Добавить студента")
        print("2. Провести лекцию")
        print("3. Проверить лабораторную")
```

```
        print("4. Провести консультацию")
        print("5. Принять экзамен")
        print("6. Выставить оценку")
        print("7. Показать студентов")
        print("0. Выход")

        choice = input("Выберите пункт: ")

        if choice == "1":
            print("1 - Бакалавр")
            print("2 - Магистр")
            student_type = input("Тип студента: ")
            name = input("Имя студента: ")

            student = StudentFactory.create_student(student_type, name)
            teacher.add_student(student)

        elif choice == "2":
            teacher.lecture()

        elif choice in ["3", "4", "5", "6"]:
            teacher.show_students()
            num = int(input("Выберите студента: ")) - 1

            if num < 0 or num >= len(teacher.students):
                print("Неверный выбор")
                continue

            student = teacher.students[num]

            if choice == "3":
                teacher.check_lab(student)

            elif choice == "4":
                teacher.consult(student)

            elif choice == "5":
                teacher.exam(student)

            elif choice == "6":
                grade = input("Введите оценку: ")
                teacher.grade(student, grade)

        elif choice == "7":
            teacher.show_students()

        elif choice == "0":
            break

        else:
            print("Неверный пункт меню")


if __name__ == "__main__":
    main()
```

Рисунки с результатами работы программы





Задание 2. ДУ автомобиля. Реализовать иерархию автомобилей для конкретных производителей и иерархию средств дистанционного управления. Автомобили должны иметь присущие им атрибуты и функции. ДУ имеет три основные функции – удаленная активация сигнализации, удаленное открытие/закрытие дверей и удаленный запуск двигателя. Эти функции должны отличаться по своим характеристикам для различных устройств ДУ.

Код программы

```
from abc import ABC, abstractmethod


class Car(ABC):

    def __init__(self, brand):
        self.brand = brand
        self.engine_started = False
        self.doors_open = False
        self.alarm_on = False

    def start_engine(self):
        self.engine_started = True
        print(f"{self.brand}: Двигатель запущен")

    def stop_engine(self):
        self.engine_started = False
        print(f"{self.brand}: Двигатель остановлен")
```

```
    def open_doors(self):
        self.doors_open = True
        print(f"{self.brand}: Двери открыты")

    def close_doors(self):
        self.doors_open = False
        print(f"{self.brand}: Двери закрыты")

    def alarm_on_func(self):
        self.alarm_on = True
        print(f"{self.brand}: Сигнализация включена")

    def alarm_off_func(self):
        self.alarm_on = False
        print(f"{self.brand}: Сигнализация отключена")


class BMW(Car):
    def __init__(self):
        super().__init__("BMW")


class Toyota(Car):
    def __init__(self):
        super().__init__("Toyota")


class Audi(Car):
    def __init__(self):
        super().__init__("Audi")


# Абстрактный пульт ДУ


class RemoteControl(ABC):

    def __init__(self, car):
        self.car = car

    @abstractmethod
    def alarm(self):
        pass

    @abstractmethod
    def doors(self):
        pass

    @abstractmethod
    def engine(self):
        pass


# Конкретные пульты


class StandardRemote(RemoteControl):

    def alarm(self):
        print("Стандартный пульт:")
        self.car.alarm_on_func()

    def doors(self):
```

```
        print("Стандартный пульт:")
        self.car.open_doors()

    def engine(self):
        print("Стандартный пульт: запуск двигателя запрещен")


class PremiumRemote(RemoteControl):

    def alarm(self):
        print("Премиум пульт:")
        self.car.alarm_on_func()

    def doors(self):
        print("Премиум пульт:")
        self.car.open_doors()

    def engine(self):
        print("Премиум пульт:")
        self.car.start_engine()


class SmartRemote(RemoteControl):

    def alarm(self):
        print("Смарт пульт:")
        self.car.alarm_on_func()
        print("Уведомление отправлено на телефон")

    def doors(self):
        print("Смарт пульт:")
        self.car.open_doors()

    def engine(self):
        print("Смарт пульт:")
        self.car.start_engine()
        print("Контроль температуры салона включен")


# Меню


def main():

    car = None
    remote = None

    while True:
        print("\nМеню:")
        print("1. Выбрать автомобиль")
        print("2. Выбрать пульт")
        print("3. Включить сигнализацию")
        print("4. Открыть двери")
        print("5. Запустить двигатель")
        print("0. Выход")

        choice = input("Выберите пункт: ")

        if choice == "1":
            print("1. BMW")
            print("2. Toyota")
            print("3. Audi")

            car_choice = input("Выберите автомобиль: ")
```

```
            if car_choice == "1":
                car = BMW()
            elif car_choice == "2":
                car = Toyota()
            elif car_choice == "3":
                car = Audi()

        elif choice == "2":

            if car is None:
                print("Сначала выберите автомобиль")
                continue

            print("1. Стандартный пульт")
            print("2. Премиум пульт")
            print("3. Смарт пульт")

            remote_choice = input("Выберите пульт: ")

            if remote_choice == "1":
                remote = StandardRemote(car)
            elif remote_choice == "2":
                remote = PremiumRemote(car)
            elif remote_choice == "3":
                remote = SmartRemote(car)

        elif choice == "3":
            if remote:
                remote.alarm()

        elif choice == "4":
            if remote:
                remote.doors()

        elif choice == "5":
            if remote:
                remote.engine()

        elif choice == "0":
            break


if __name__ == "__main__":
    main()
```

Рисунки с результатами работы программы

```
ПРОБЛЕМЫ    ВЫХОДНЫЕ ДАННЫЕ    КОНСОЛЬ ОТЛАДКИ

Меню:
1. Выбрать автомобиль
2. Выбрать пульт
3. Включить сигнализацию
4. Открыть двери
5. Запустить двигатель
0. Выход
Выберите пункт: 1
1. BMW
2. Toyota
3. Audi
Выберите автомобиль: 1

Меню:
1. Выбрать автомобиль
2. Выбрать пульт
3. Включить сигнализацию
4. Открыть двери
5. Запустить двигатель
0. Выход
Выберите пункт: 2
1. Стандартный пульт
2. Премиум пульт
3. Смарт пульт
Выберите пульт: 3
```

```
ПРОБЛЕМЫ    ВЫХОДНЫЕ ДАННЫЕ    КОНСОЛЬ ОТЛАДКИ

Меню:
1. Выбрать автомобиль
2. Выбрать пульт
3. Включить сигнализацию
4. Открыть двери
5. Запустить двигатель
0. Выход
Выберите пункт: 3
Смарт пульт:
BMW: Сигнализация включена
Уведомление отправлено на телефон

Меню:
1. Выбрать автомобиль
2. Выбрать пульт
3. Включить сигнализацию
4. Открыть двери
5. Запустить двигатель
0. Выход
Выберите пункт: 4
Смарт пульт:
BMW: Двери открыты

Меню:
1. Выбрать автомобиль
2. Выбрать пульт
3. Включить сигнализацию
4. Открыть двери
5. Запустить двигатель
0. Выход
Выберите пункт: 5
Смарт пульт:
BMW: Двигатель запущен
Контроль температуры салона включен
```

Задание 3. Проект «Пиццерия». Реализовать формирование заказ(а)ов, их отмену, а также повторный заказ с теми же самыми позициями.

Код программы

```
from abc import ABC, abstractmethod

# Получатель (Receiver)


class Pizzeria:

    def create_order(self, items):
        print("Заказ создан:")
        for item in items:
            print(f"- {item}")

    def cancel_order(self):
        print("Заказ отменен")


# Интерфейс команды


class Command(ABC):

    @abstractmethod
```

```
    def execute(self):
        pass

    @abstractmethod
    def undo(self):
        pass


# Конкретная команда


class OrderCommand(Command):

    def __init__(self, pizzeria, items):
        self.pizzeria = pizzeria
        self.items = items

    def execute(self):
        self.pizzeria.create_order(self.items)

    def undo(self):
        self.pizzeria.cancel_order()


# Инициатор


class Waiter:

    def __init__(self):
        self.history = []
        self.last_command = None

    def take_order(self, command):
        self.last_command = command
        self.history.append(command)
        command.execute()

    def cancel_last(self):
        if self.history:
            command = self.history.pop()
            command.undo()
        else:
            print("Нет заказов для отмены")

    def repeat_last(self):
        if self.last_command:
            print("Повтор последнего заказа:")
            self.last_command.execute()
        else:
            print("Нет заказа для повторения")


# Меню


def main():

    pizzeria = Pizzeria()
    waiter = Waiter()

    while True:
        print("\nМеню:")
        print("1. Сделать заказ")
```

```
        print("2. Отменить последний заказ")
        print("3. Повторить последний заказ")
        print("0. Выход")

        choice = input("Выберите пункт: ")

        if choice == "1":

            items = []

            while True:
                print("\nДобавить позицию:")
                print("1. Пицца Маргарита")
                print("2. Пицца Пепперони")
                print("3. Кола")
                print("4. Завершить заказ")

                item_choice = input("Выбор: ")

                if item_choice == "1":
                    items.append("Пицца Маргарита")

                elif item_choice == "2":
                    items.append("Пицца Пепперони")

                elif item_choice == "3":
                    items.append("Кола")

                elif item_choice == "4":
                    break

            if items:
                command = OrderCommand(pizzeria, items)
                waiter.take_order(command)

        elif choice == "2":
            waiter.cancel_last()

        elif choice == "3":
            waiter.repeat_last()

        elif choice == "0":
            break


if __name__ == "__main__":
    main()
```

Рисунки с результатами работы программы

```
ПРОБЛЕМЫ    ВЫХОДНЫЕ ДАННЫЕ    КОНСОЛЬ ОТЛАДКИ

Меню:
1. Сделать заказ
2. Отменить последний заказ
3. Повторить последний заказ
0. Выход
Выберите пункт: 1

Добавить позицию:
1. Пицца Маргарита
2. Пицца Пепперони
3. Кола
4. Завершить заказ
Выбор: 3

Добавить позицию:
1. Пицца Маргарита
2. Пицца Пепперони
3. Кола
4. Завершить заказ
Выбор: 4
Заказ создан:
- Кола
```

```
Меню:
1. Сделать заказ
2. Отменить последний заказ
3. Повторить последний заказ
0. Выход
Выберите пункт: 3
Повтор последнего заказа:
Заказ создан:
- Кола

Меню:
1. Сделать заказ
2. Отменить последний заказ
3. Повторить последний заказ
0. Выход
Выберите пункт: 2
Заказ отменен

Меню:
1. Сделать заказ
2. Отменить последний заказ
3. Повторить последний заказ
0. Выход
Выберите пункт:
```

Вывод: приобрел навыки применения паттернов проектирования при решении практических задач с использованием языка Python.